



What is Parity's ink!?

On the occasion of ink! surpassing 1,000 stars on Github, we take a deep dive into the smart contract programming language. ink! is a...

Michi Müller

Core Developer @ Parity Technologies @ Parity Technologies

September 19, 2022

13 Min Read



ink! is a programming language for smart contracts — one of several that blockchains built with the Substrate framework can choose from. It's an opinionated language that we at Parity have built by extending the popular Rust programming language with functionality needed to make it smart contract compatible.

Thank you for helping ink! surpass 1,000 GitHub stars!

The ink! repository recently surpassed one thousand stars on GitHub. It's now the third most starred Parity repository on GitHub, after Substrate and Polkadot. We want to say a big "Thank you!" to everyone who contributed to making this happen! Over the last few years we've seen contributions in all forms bringing the project to where it stands today. Thank you for writing your contracts in ink!, for creating issues and pull requests, for providing us with good and constructive feedback, for answering questions on Substrate StackExchange, for creating third-party tooling for ink!, for writing blog posts about us, and for holding workshops!

A lot has happened over the last few years in the ink! world. The first commit was in December, 2018 and our current version is v3.3.1. We're working hard on shipping the next major iteration with v4.0 soon. An open point that comes up regularly is that we've never written a blog post explaining from start to finish what ink! is and how it ties into Substrate and Polkadot. Hence this occasion is a great opportunity to do just that!

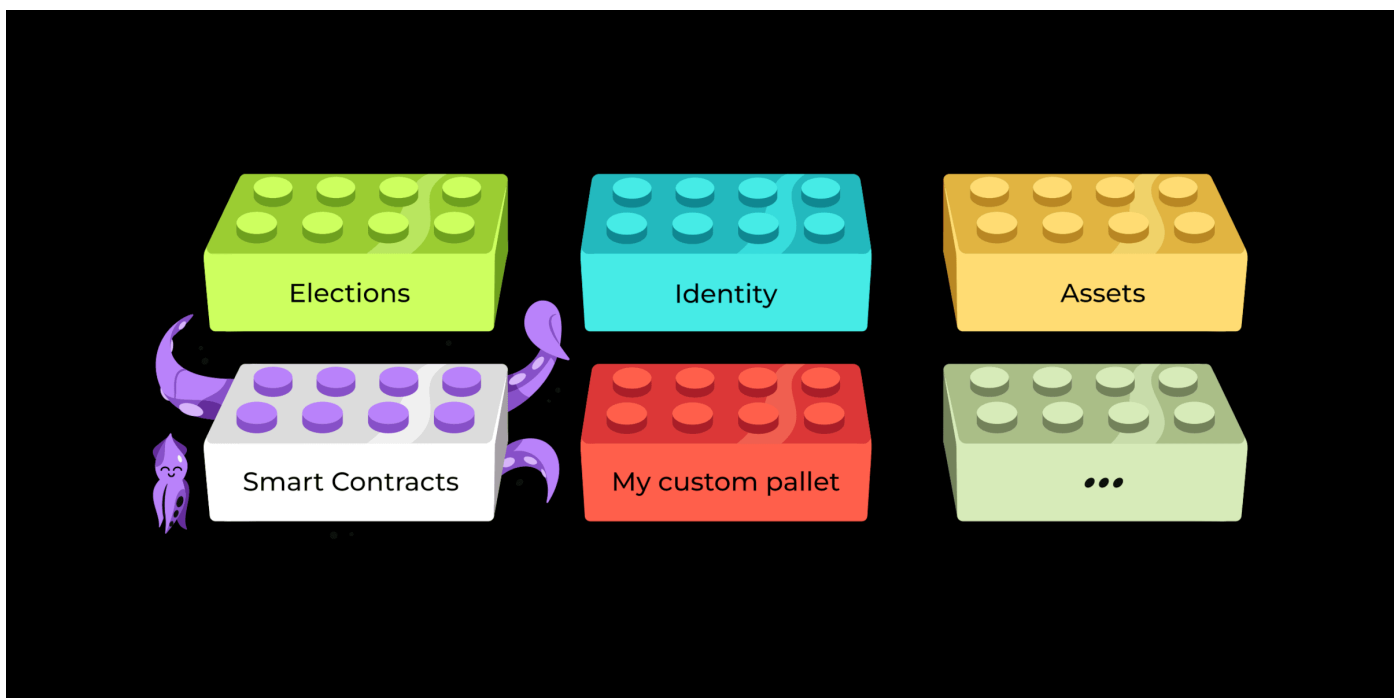
How does ink! relate to Substrate?

Before we can talk about ink! we first need to clarify what Substrate and its Contracts pallet (pallet-contracts) are. Substrate is a framework for building blockchains, which can be standalone blockchains or blockchains connected to Kusama or Polkadot as so-called parachains. Substrate contains a number of modules, which in Substrate terminology are called pallets. Substrate comes with a set of pallets for many requirements modern blockchains typically have — staking, fungible tokens, non-fungible tokens, governance, etc

Substrate also ships with a module for smart contracts called the Contracts pallet. If a parachain is developed in Substrate, it can easily add smart contract functionality by including this pallet.

How does ink! come into play here? ink! is a programming language, specifically it is an embedded domain-specific language (eDSL) for the popular Rust programming language. This means that you can use all the normal Rust syntax plus some specifics that we added to make the language suitable for the smart contract world. The Contracts pallet takes these ink! contracts and executes them in a safe manner. So in short:

With ink! you can write smart contracts in Rust for blockchains built with Substrate that include the Contracts pallet.



Smart contracts vs. parachains

One of the first questions we typically get when somebody learns about Substrate, Polkadot, or Kusama is when to develop a parachain vs. when to develop a smart contract.

The distinction here is that, in the context of Polkadot and Kusama, a parachain leases a slot for a couple of months for up to two years. The deal with a lease is that the parachain gets a fixed slot for executing its business logic (typically referred to as its state transition function) and can persist its modified state in a block. In Substrate terminology this state transition function is called the chain's runtime.

The distinction to other ecosystems here is that, in the context of Polkadot, parachains and smart contracts exist at different layers of the stack: smart contracts sit on top of

parachains. Parachains would usually be described as layer-1 blockchains — except for that they don't have to build their own security, and are upgradable and interoperable.

It's noteworthy that a parachain's state transition function doesn't get further validated — it's up to the parachain how it utilizes its slot time. Once a parachain secures a slot by bonding tokens (or sourcing them from their community via [crowdloan](#)), the slot is essentially pre-paid, with no additional fees required for executing the chain's business logic. This means the parachain can build its own (blockchain) world! For example, it can decide on how transaction fees are charged, or even if transaction fees are charged at all. These options are crucial when building new or more user-friendly business models.

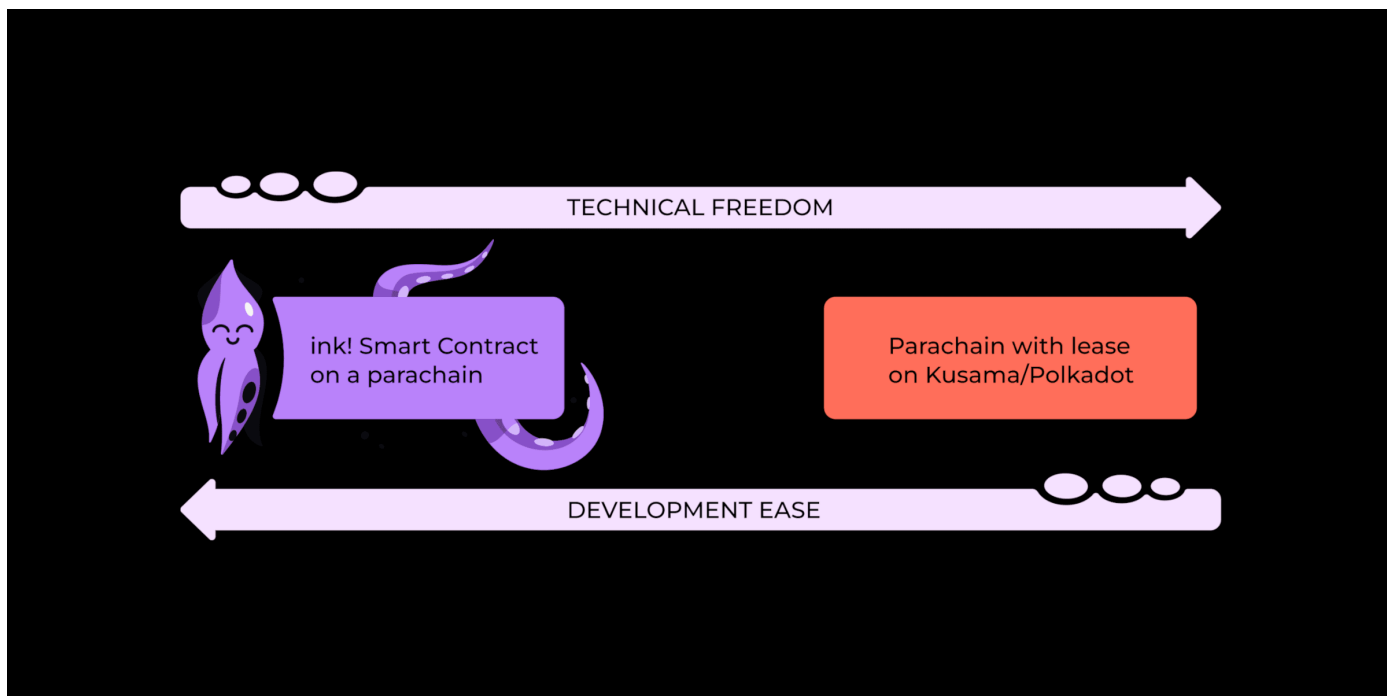
Other distinguishing factors between parachains that we observe in the wild are differences in how the chain's governance or crypto-economics work. There are some constraints on how the parachain can build its world though. Like physics in the real world, it has to adhere to certain ground rules. For Polkadot and Kusama that's for example the consensus algorithm for the relay chain to communicate with the parachain. From those ground rules the advantages of Polkadot and Kusama emerge. Advantages like the aforementioned shared security, cross-chain communication, or guaranteed execution slot time.

For smart contracts, on the other hand, an existing parachain has to include the Contracts pallet for users to deploy smart contracts. The deployed smart contract is always untrusted code. Anyone (or any program) that has tokens of the chain can upload a smart contract without requiring permission. Smart contracts allow permissionless deployment of untrusted programs on a blockchain. The Contracts pallet has to assume that these programs are adversarial, it has to put a number of safety pillars in place to ensure that the contract can not e.g. stall the chain or cause state corruption of other contracts. For the Contracts pallet those safety pillars include mechanisms like gas metering or deposits for storing data on-chain.

To restate this important distinction: developing a parachain runtime is different from developing a smart contract — a smart contract sits on top of a parachain.

The trade-off is that with a parachain one has the freedom to decide on (nearly) all the rules that make up the parachain. With a smart contract one is constrained by what the chain allows and the safety pillars that necessarily have to be in place.

A smart contract on the other hand has less friction for developing and deploying it. Developers don't have to think about governance, crypto-economics, etc. One just needs a few tokens and can go on their merry way deploying a smart contract. It's as simple as that.



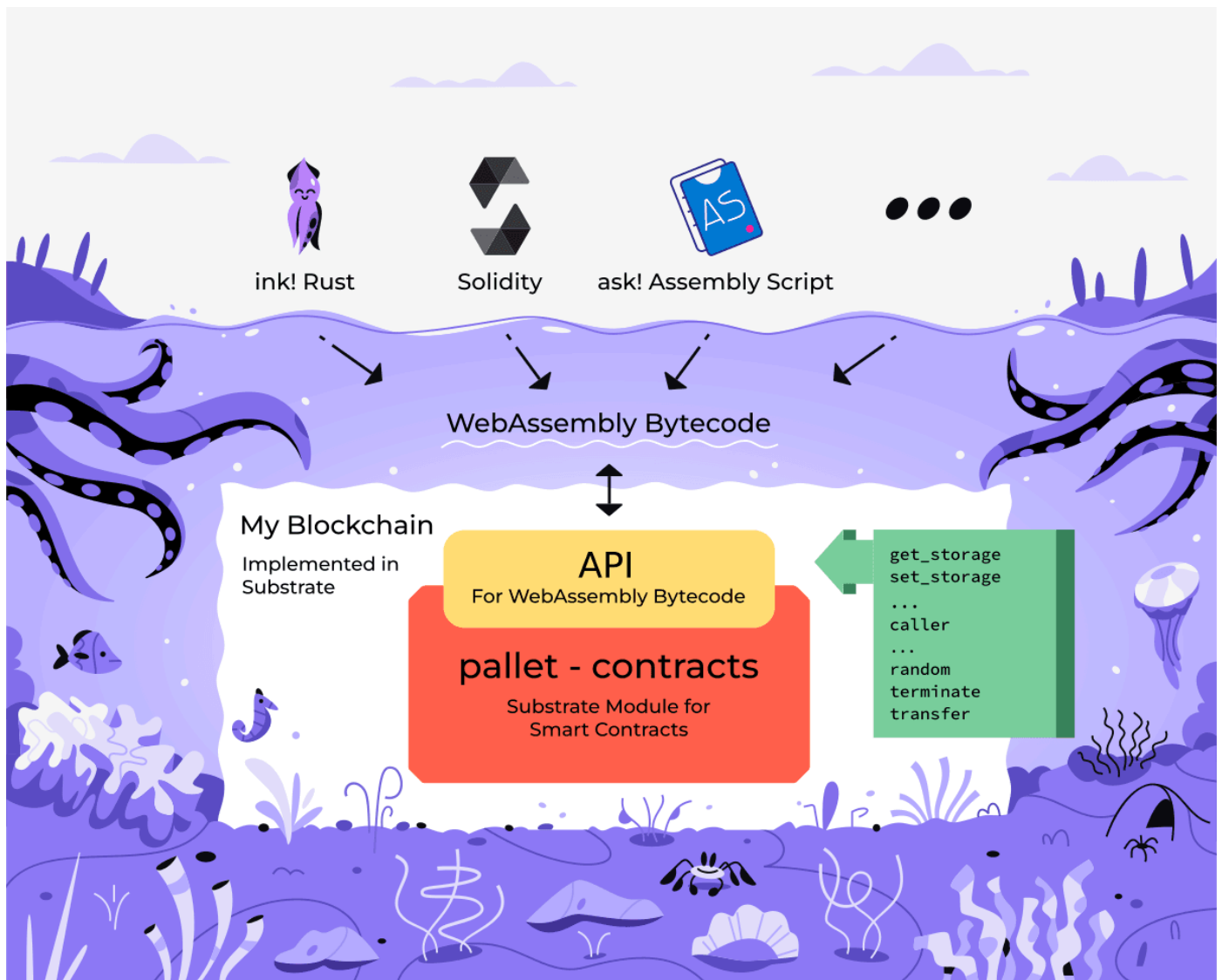
How does the Contracts pallet work?

We intentionally designed the Contracts pallet in a way that it is decoupled from the language used to write smart contracts. The pallet is only the execution environment and it takes WebAssembly files as input. Smart contracts for this pallet have to be compiled to the WebAssembly (Wasm) target architecture.

For contract developers this means they can use ink! for writing smart contracts, but can also decide on other languages. Right now three languages to choose from exist:

- [Parity's ink! for Rust](#)
- [ask! for AssemblyScript](#)
- [The Solang compiler for Solidity](#)

It's not hard to add new languages. There just needs to be a compiler for the language down to WebAssembly, then it's possible to implement the Contracts pallet API. This API at the moment consists of about 15-20 functions for anything a smart contract may desire: storage access, cryptographic functionality, environmental information like block numbers, access to functions for getting random numbers or to self-terminate the contract, etc. Not all of those have to be implemented in the language — the ink! "Hello, World!" requires just six API functions. The following schema depicts this relationship:



We think this design is more future-proof than some architectures found in competing ecosystems. There is no tight coupling between language and execution environment. WebAssembly is an industry standard and a multitude of programming languages can nowadays be compiled down to WebAssembly. If in, say ten years time, researchers come up with an innovative language for writing smart contracts (or a subset of an existing language) then as long as there is a WebAssembly compiler it will be easy to make this language compatible with the Contracts pallet.

Why include the Contracts pallet on a parachain?

There are a couple use cases for including smart contract functionality on a parachain. We distinguish three big ones:

Use Case 1: Smart Contracts as “first-class citizens”

The most obvious use case is a parachain which provides smart contracts as a “first-class citizen”, meaning smart contracts are the central value proposition of the chain.

Those chains typically take the off-the-shelf Contracts pallet and create some additional innovation on top of it. Examples of this are:

- Astar: built a layer on top of the Contracts pallet so that contract developers can earn a passive income whenever their contracts are being used.
- Phala: utilizes the Contracts pallet in a trusted execution environment, enabling confidential smart contract execution and interoperability.
- Aleph Zero: uses the Contracts pallet in a zero-knowledge context.
- t3rn: uses the Contracts pallet as a building block to enable multichain execution for smart contracts.

Use Case 2: Smart Contracts as “second-class citizens”

There is another not so obvious use case for the Contracts pallet: smart contracts as “second-class citizens” on an existing chain. By this we mean that the central value proposition of the chain has nothing to do with smart contracts, but it still includes them as an add-on.

We provide an API (called chain extensions) with which a parachain can expose certain parts of its business logic to smart contract developers. Through this, smart contract developers can utilize the business logic primitives of the chain to build a new application on top of it. Think for example of a decentralized exchange blockchain. This chain would in its simplest form have an order book to place bids and asks — there is no need for taking untrusted, turing complete, programs from the outside. The parachain could decide to expose the order book into smart contracts though, giving external developers the option of building new applications that utilize the order book. For example, to upload trading algorithms as smart contracts to the chain

Smart contracts here are an opportunity to help increase user engagement. And the billing for utilizing the chain comes already built-in with the pallet — users have to pay gas fees for the execution of their smart contract.

Use Case 3: Smart Contracts as a first step into Polkadot or Kusama

A third big use case for the Contracts pallet is to prototype an idea as a proof-of-concept smart contract before leasing a dedicated parachain slot on Polkadot or Kusama.

The time to develop a smart contract and deploy it is shorter than the onboarding story for a parachain. One can deploy a proof-of-concept smart contract first, see if it gains traction and the idea holds up to the real world. Only subsequently, once there is a need for e.g. cheaper transaction fees, more efficient execution, or a governance mechanism for the

community, the smart contract could be migrated to a dedicated parachain runtime with its own slot. ink! contracts and Substrate runtimes are both written in Rust and share similar primitives, this enables a clear path for a project to graduate from a smart contract to its own runtime. Developers can reuse large parts of their code, their tests, as well as frontend and client code.



A simple ink! smart contract

ink! is really just Rust, that's our overarching goal. We aim to be minimally invasive, enabling developers to use everything that they also can use for "normal" Rust — IDEs, `cargo fmt`, `cargo clippy`, code snippets, the crates.io ecosystem, etc.

In the following picture you can see a simple ink! contract. The contract here holds one boolean value in its storage. Once the contract is created it sets the boolean to `true`. The contract exposes two functions: one to read the current value of the boolean (`fn get()`) and one to switch the value to its opposite boolean value (`fn flip()`).

```

#[ink(contract)]
mod my_contract {
    #[ink(storage)]
    struct MyContract {
        value: bool,
    }

    impl MyContract {
        #[ink(constructor)]
        pub fn new() → Self {
            MyContract { value: true }
        }

        #[ink(message)]
        pub fn get(&self) → bool {
            self.value
        }

        #[ink(message)]
        pub fn flip(&mut self) {
            self.value = !self.value;
        }
    }
}

```

The colored lines are ink!-specific annotations in the code, the rest is just normal Rust syntax. These annotations abstract away from what needs to happen under the hood in order to make the program compatible to be executed on-chain.

For our current version 3.3.1, unit and integration tests can also be written as they are in “normal” Rust:

```

#[ink::test]
fn default_works() {
    let flipper = Flipper::default();
    assert_eq!(flipper.get(), false);
}

```

The `#[ink::test]` annotation here has the effect of executing this test in a mocked blockchain environment. This enables developers to mock e.g. the value that is transferred to a contract, the caller executing a contract, the block number, etc. For example, you can use `ink_env::test::set_value_transferred` within an `#[ink::test]` to mock the value (i.e. tokens) that are sent to a contract. You can see the full list of `ink_env` functions in our [crate documentation](#).

cargo-contract - the Swiss army knife for ink! contracts

For building ink! smart contracts you could just use the normal cargo build workflow for building Rust programs. You would have to add a number of arguments for the command

though to make it work on-chain. We've created a tool that already chooses the optimal set of flags for you: `cargo-contract`. It's a command-line tool that mirrors cargo. You can think of it like a Swiss army knife for ink! smart contracts, it can do much more besides just building a contract, but we'll talk about that later.

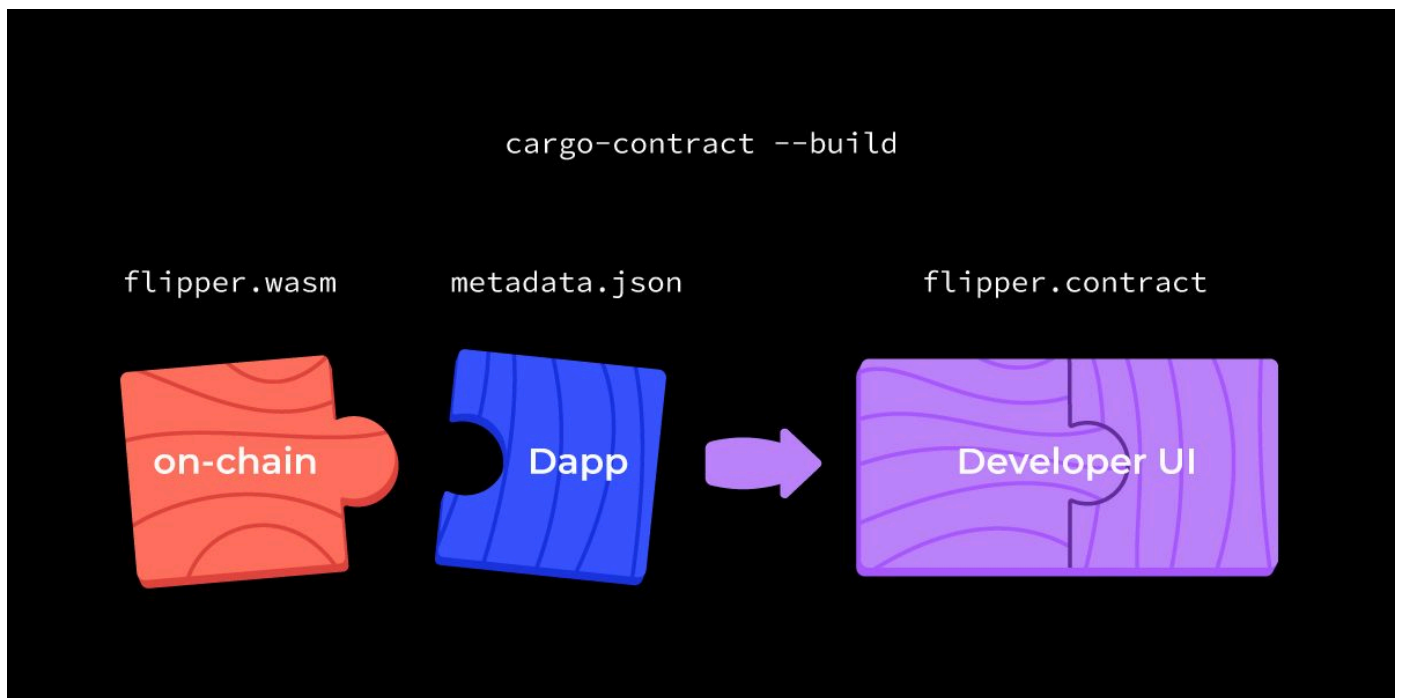
For building contracts you use `cargo contract build`. Note that the typical Rust cargo build behavior is that you need to supply `--release` if you want the smallest possible binary size, same with `cargo contract build --release`. If you run this command you'll see that `cargo-contract` executes the "normal" `cargo build` on your contract, but it does some more steps as well. The three most important additional steps are:

- It runs a linter for ink! contracts. This linter works analog to Rust's `clippy`, it checks your contract for idiomatic use of ink!. We are constantly improving this linting tool and will add detections of common security pitfalls in the future.
- It post-processes and compresses your contract's binary. This is done to reduce the costs for users deploying a contract, as well as the fees for users interacting with the contract. The contract size also correlates to the throughput a chain can achieve and contributes to the chain footprint.
- It generates metadata for the contract. With the term metadata we refer to any information that is needed for interacting with the contract's binary. The binary itself will be just a WebAssembly blob — byte code of the contract that without further information cannot be interacted with. In order to know, for example, which functions the contract exposes and what arguments those functions take one needs to have the metadata. You might also know this concept under the term ABI from other blockchains. In our case the metadata contains a bit more than just the ABI though, it also contains information on e.g. how the contract stores its data, which is helpful for off-chain tooling (e.g. a block explorer) or human readable documentation about the contract functions.

As a result of executing `cargo contract build` three files are created:

- `my_contract.contract`: a JSON file that contains the contract's WebAssembly blob in hex encoding plus the contract's metadata.
- `metadata.json`: a JSON file that contains just the contract's metadata, without the WebAssembly blob.
- `my_contract.wasm`: the contract's WebAssembly blob.

Each of those files has a different use case:



The WebAssembly is the only file that is actually stored on-chain. Storing data on-chain should only be done for data that strictly needs to be on-chain. Anything that is unnecessary on-chain would still incur costs for the users and bloat up the chain footprint. The metadata is not necessary to be on-chain, a dapp or frontend can contain the hardcoded metadata in order to determine how to interact with the contract.

The `*.contract` bundle is only needed if you are developing a smart contract, then you can use this file with a Developer UI. Developer UI's give you the ability of deploying contracts, interacting with them, and debugging them.

Development Environments

A number of parachains have developed custom tooling that provides a more context-specific angle on the context in which they utilize the Contracts pallet and ink!. We mentioned some of those teams in the beginning of this post. For more information see our [awesome-ink](#) repository.

We, as the team developing ink! and the Contracts pallet, provide a couple of handy tools as well:

- [substrate-contracts-node](#) This repository contains Substrate's node-template configured to include pallet-contracts. This node is tracking Substrate's master branch and has been modified to make it a great fit for development and testing. For example, it does not have any fixed block time, everything is processed immediately. This comes at the cost of making the node unsuitable for production use, but great for scripting, testing, or a Continuous Integration (CI) environment. If you are looking for production templates take a

look at Substrate's [node](#) or the How-To guide in Substrate's documentation on how to add the pallets—contracts ([link here](#)).

- We maintain a testnet named plainly Contracts on Rococo. Rococo is a testnet for anything Polkadot and Kusama. Many parachains are connected to Rococo with their own testnets. You can read more about how to deploy on our testnet in [our documentation](#).

There's also a number of community testnets, you can find some in our [awesome-ink repository](#).

For Developer UI's there are currently three choices:

- [Contracts UI](#): Ideal for beginners, gives helpful hints and assists developers.
- [polkadot.js](#): Advanced interface which gives developers flexibility, at the cost of user friendliness.
- [ink! playground](#): A playground to try out ink! in the browser or share permanent links to code snippets. A very handy use is if you are asking questions on the [Substrate StackExchange](#): you can share code with e.g. a problem you are facing.
- [cargo-contract](#): If you are looking to deploy contracts from the command-line, call them, decode their output, etc. then this tool provides a handy option to do all of that from a command-line interface. The tool is also perfect for scripting or CI integrations.

Why choose this route over a Substrate Ethereum compatibility layer?

Besides the Contracts pallet, two other popular options for smart contracts in Substrate are pallet-evm and [Frontier](#). Both are Ethereum compatibility layers for Substrate.

There are a number of advantages for choosing the route of ink! and the Contracts pallet over the EVM one. To summarize a few which were detailed in this article:

- ink! is just Rust — you can use all the normal Rust tooling: [clippy](#), [crates.io](#), IDE's, etc.
- Rust is a language that incorporates years of language research, it's safe and fast. On top of that we took major learnings from older smart contract languages (like Solidity) and incorporated them into the ink! language design. We've chosen more sensible default behavior, such as disabling re-entrancy by default or making functions private by default.
- Rust is an amazing language to work with, it has been crowned the most loved programming language for seven years in a row on StackOverflow ([source](#)).
- If you are a company and looking to hire smart contract developers, you can hire from the Rust ecosystem, which is much larger than the niche of e.g. Solidity developers.
- ink! is native to Substrate, we use similar primitives, like the same type system.
- There is a clear migration path to graduate from a contract to a parachain. Since ink! and Substrate are both Rust, developers can reuse large parts of their code, their tests, as well

as frontend and client code.

- WebAssembly is an industry standard, it's not just used in the blockchain world. It is continually improved by major companies such as Google, Apple, Microsoft, Mozilla, and Facebook. Going forward we will profit from all improvements to the standard and its implementations. This is a big thing. WebAssembly is mostly used for the web, which has similar requirements as blockchains — safe and fast.
- WebAssembly expands the family of languages available to smart contract developers to include Rust, C/C++, C#, Typescript, Haxe, Kotlin, etc. This means you can write smart contracts in whichever language you're familiar with. We think this design is more future proof than a tight coupling of language and execution architecture.

We support interoperability with legacy Solidity codebases: the HyperLedger project [Solang](#) compiles Solidity for the Contracts pallet.

Where to learn more?

- Our documentation portal can be found at [use.ink](#), it contains the most comprehensive information on ink!.
- Since ink! is just Rust, you can also check out the crate docs. We've linked relevant crate docs in our readme [here](#), e.g. the crate documentation for data structures is linked there.
- We have a number of contract examples in our repository [here](#). Those cover a broad number of typical use-cases: ERC-20, upgradeable contracts, etc.
- If you are looking for a quickstart, [ink!'s Guided Tutorial for Beginners](#) provides a great starting point.
- In case you are interested in the inner workings of ink!, we explain those in our [ARCHITECTURE.md](#) file.
- For the Contracts pallet the [README.md](#) provides a starting point.

Going forward

Our intention with this article was to provide a complete picture on what ink! is all about. The next big step for ink! is v4.0.0 --- our next major iteration for which we already released an alpha. This release will bring major improvements to contract sizes, the developer experience and native End-to-End testing. Besides this, a number of parachain teams are on the verge of launching the Contracts pallet and ink! to Kusama or Polkadot. Things are staying excited and we're very much looking forward to the next one thousand stars.

In case you are left with any questions, please don't hesitate to reach out. The best way to do so is to ask on the [Substrate StackExchange](#) or via <https://info.polkadot.network/contact>. If you are already building, open an issue in the relevant repository.



Company

[Technology](#)

[Culture](#)

[Careers](#)

[About Us](#)

[Blog](#)

[Contact](#)

Opportunities

[Parity Data Dashboard](#)

[GitHub](#)

[Bug Bounty](#)

[Blockchain Academy](#)

© 2024 Parity Technologies

[Manage Cookies](#) [Terms of Service](#) [Privacy Policy](#) [Applicant Notice](#)